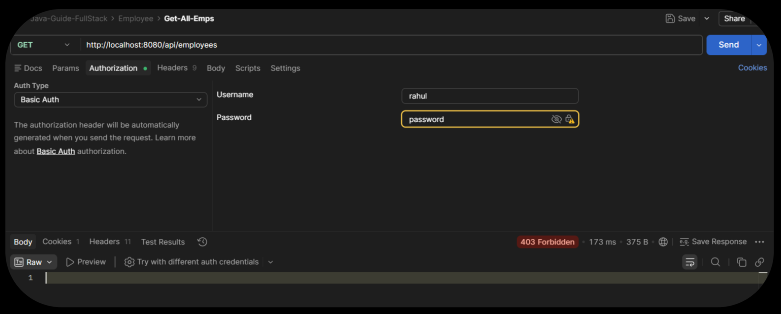
Today 4:24 PM




I am working a full stack crud app using react and Spring boot. Now I have started adding Spring Security to the backend app only like securing rest apis and testing it from postman.
But I am facing an issue.

I have added the required spring security dependency.
For now configured one dummy user using application.properties file and configure the SecurityFilterChain as below

Show more

Your configuration is **almost correct**, but there is one important thing missing.

The fact that you're getting **403 Forbidden** instead of **401 Unauthorized** is actually a very useful clue.

## First, your configuration

Java

```java
@Configuration
public class SpringSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exce

        httpSecurity.csrf(AbstractHttpConfigurer::disable);

        httpSecurity.authorizeHttpRequests(request ->
                request.anyRequest().authenticated());

        return httpSecurity.build();
    }
}
```

This simply says:

- Disable CSRF
- Every request must be authenticated

It **does NOT tell Spring Security which authentication mechanism to use.**

---

## You are using Basic Authentication in Postman

In Postman you selected

```
Authorization
    Basic Auth

username = rahul
password = password
```

For Spring Security to process this Authorization header, you must enable HTTP Basic authentication.

Add:

```java
import org.springframework.security.config.Customizer;

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth ->
                    auth.anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

Notice this line:

```java
.httpBasic(Customizer.withDefaults());
```

Without it, Spring Security won't use Basic Authentication.

## Also make sure the method signature is correct

It should be

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
```

because

```java
http.build();
```

throws an exception.

---

## Next, verify your application.properties

```properties
spring.security.user.name=rahul
spring.security.user.password=password
```

These are correct.

---

## If it STILL returns 403

There are several possibilities.

### 1. Is your SecurityFilterChain actually being picked up?

Add

```java
System.out.println("Security Config Loaded");
```

inside the bean.

If nothing prints, your configuration class isn't being scanned.

---

## 2. Turn on Spring Security logs

Add

```properties
logging.level.org.springframework.security=TRACE
```

Then restart.

When you send the request you'll see something like

```
Authenticated user rahul
```

or

```
No AuthenticationProvider found
```

or

```
Access Denied
```

Those logs usually reveal the exact cause.

---

## 3. Check the response body

Your screenshot only shows

```
403 Forbidden
```

What is inside the response body?

Sometimes Spring returns something like

```JSON
{
    "timestamp": "...",
    "status":403,
    "error":"Forbidden",
    "path":"/api/employees"
}
```

or

```
Invalid CSRF Token
```

The body is important.

---

### 4. Are you using Spring Boot 3.x?

Which version?

For example:

```
Spring Boot 3.5.x
```

or

```
Spring Boot 4.x
```

The configuration differs slightly.

---

### 5. Is there another SecurityFilterChain?

If another configuration exists, yours may not be used.

Search your project for

```java
SecurityFilterChain
```

or

```java
@EnableWebSecurity
```

There should usually be only one filter chain while you're learning.

---

## One thing I noticed

Your code snippet is missing

```java
```

```
throws Exception
```

It should be

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http.csrf(AbstractHttpConfigurer::disable);

    http.authorizeHttpRequests(auth ->
            auth.anyRequest().authenticated());

    http.httpBasic(Customizer.withDefaults());

    return http.build();
}
```

This is the minimal configuration that should work with:

```
Username : rahul
Password : password
```

in Postman.

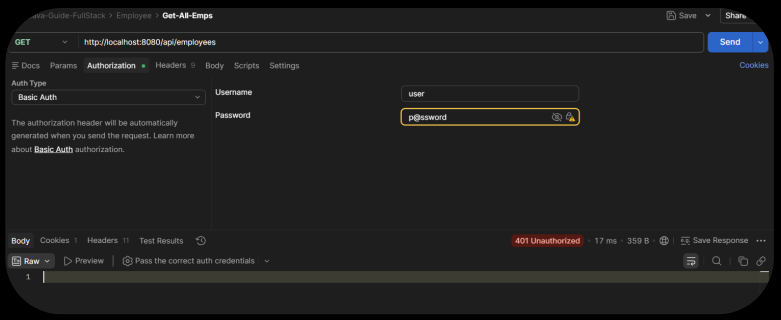
---

## If it still doesn't work

Please share these four things:

1. Your complete `pom.xml`
2. Your Spring Boot version
3. The complete response body from Postman (not just the 403 status)
4. The Spring Boot console logs after sending the request (especially with `logging.level.org.springframework.security=TRACE` enabled)

With those, it should be possible to pinpoint the exact reason for the 403.




Now I added the InMemory Users, but now not able to call the API from the postman.

```
package
com.rahul.learning.javaguide.emsbackend.configs;

import lombok.extern.log4j.Log4j2;
import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configurat
```

Show more

This is an excellent question. You're now thinking like someone designing the application rather than just getting it to work.

The answer is:

> **Yes, the `Role` table has to be populated somehow.** But **how** you do that depends on your application's requirements.

There are several common approaches.

---

## Option 1: Insert roles manually (Most common for learning)

Suppose your application only has two roles:

```
ROLE_USER
ROLE_ADMIN
```

You insert them once into the database.

```sql
INSERT INTO roles(name) VALUES ('ROLE_USER');

INSERT INTO roles(name) VALUES ('ROLE_ADMIN');
```

After that, you never insert them again.

When a new user registers:

```java
Role role = roleRepository.findByName("ROLE_USER");

user.getRoles().add(role);

userRepository.save(user);
```

Notice:

- `Role` is **read**
- `User` is **saved**
- No new role is created

This is how most applications work.

---

## Option 2: Populate roles automatically on application startup ⭐ (My recommendation)

This is my favorite approach for applications like yours.

Create a small component that runs once when the application starts.

```java
@Component
public class RoleSeeder implements CommandLineRunner {

    @Override
    public void run(String... args) {
        ...
    }
}
```

Inside it:

Java

```java
if(roleRepository.findByName("ROLE_USER") == null){
    roleRepository.save(new Role("ROLE_USER"));
}

if(roleRepository.findByName("ROLE_ADMIN") == null){
    roleRepository.save(new Role("ROLE_ADMIN"));
}
```

Now every time your application starts:

- if the roles don't exist → insert them
- if they already exist → do nothing

This works very well during development.

---

## Option 3: Use `data.sql`

Spring Boot can execute SQL automatically.

Create

```
src/main/resources/data.sql
```

SQL

```sql
INSERT INTO roles(name)
VALUES ('ROLE_USER');

INSERT INTO roles(name)
VALUES ('ROLE_ADMIN');
```

When the application starts,

Spring inserts them automatically.

Very simple.

---

## Option 4: Create Role APIs

Yes, you can create

```
RoleRepository
```

```
↓
RoleService
↓
RoleController
```

with endpoints like

```
POST /roles
GET /roles
DELETE /roles
```

But ask yourself:

> Will an admin ever create new roles from the UI?

For your Employee Management System,

probably not.

The roles are fixed.

```
ROLE_USER
ROLE_ADMIN
```

So creating CRUD APIs for roles is unnecessary.

---

## Why we usually don't create roles every time a user is created

Imagine 100 employees.

If you create a new role every time,

your database becomes

| role_id | name |
|---|---|
| 1 | ROLE_USER |
| 2 | ROLE_USER |

| role_id | name |
| --- | --- |
| 3 | ROLE_USER |
| ... | ... |

This defeats the purpose of a role table.

Instead,

| role_id | name |
| --- | --- |
| 1 | ROLE_USER |
| 2 | ROLE_ADMIN |

and the join table becomes

| user_id | role_id |
| --- | --- |
| 1 | 1 |
| 2 | 1 |
| 3 | 2 |
| 4 | 1 |
| 5 | 1 |

This is normalized and scalable.

---

## So do I need a RoleRepository?

**Yes.**

Even if you never expose role APIs, you'll almost certainly have a `RoleRepository`.

Something like:

```java
public interface RoleRepository extends JpaRepository<Role, Long> {
```

```java
    Optional<Role> findByName(String name);

}
```

You'll use it when creating users:

```java
Role userRole = roleRepository
        .findByName("ROLE_USER")
        .orElseThrow(() -> new RuntimeException("Role not found"));

user.getRoles().add(userRole);

userRepository.save(user);
```

Notice that you're **reading** an existing role, not creating one.

---

## What I would do for your project

Since you're building this project to learn Spring Security, here's the architecture I'd recommend:

```
Role Entity
        |
        ▼
RoleRepository
        |
        ▼
RoleSeeder (CommandLineRunner)
        |
        ▼
Database
```

Then, when registering a user:

```
Register User
        |
        ▼
Find ROLE_USER
        |
        ▼
Assign Role
        |
        ▼
Save User
```

No `RoleService` or `RoleController` is needed unless you later decide that administrators should be able to create and manage roles dynamically.

---

### My recommendation

For your current project, I would:

- ✅ Create a `RoleRepository`.
- ✅ Seed `ROLE_USER` and `ROLE_ADMIN` automatically at startup using a `CommandLineRunner` (or `ApplicationRunner`).
- ✅ Do **not** create `RoleController` or `RoleService` yet.
- ✅ Whenever you create a new user, fetch the existing role from the database and attach it.

This mirrors how many real-world Spring Boot applications manage fixed roles while keeping the design simple and maintainable.

Okay, great, will follow yours.
I forgot to ask one thing; what is the difference between joinColumn and inverseJoinColumn that I used during ManyToMany mapping?

I'm glad you asked this because `joinColumns` and `inverseJoinColumns` confuse almost everyone when they first learn JPA. Once you understand the perspective from which Hibernate looks at the relationship, it becomes very simple.

Let's use your exact example.

Java

```java
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "users_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
private Set<Role> roles;
```

---

# First, imagine the database

```
users
------
user_id
username

roles
------
role_id
name

users_roles
------------
user_id
role_id
```

The `users_roles` table simply stores relationships.

Example:

| user_id | role_id |
|---|---|
| 1 | 1 |
| 1 | 2 |
| 2 | 2 |

means

```
User 1
   ├── Role 1
   └── Role 2

User 2
   └── Role 2
```

---

# The important question is:

> **From whose perspective are we looking?**

Your mapping is inside the **User** class.

```java
public class User {

    private Set<Role> roles;

}
```

So Hibernate says:

> "I am currently mapping the **User** entity."

Everything is now viewed from the **User's perspective**.

---

## What does `joinColumns` mean?

```java
joinColumns = @JoinColumn(name = "user_id")
```

This means:

> **In the join table, which column represents THIS entity?**

Since this entity is **User**,

```
users_roles

user_id   ← THIS entity (User)
role_id
```

So `joinColumns` always refers to **the owner/current entity**.

Think of it as:

```
I'm User.

Which column belongs to me?

Answer:

user_id
```

---

## What does `inverseJoinColumns` mean?

Now Hibernate asks:

> "What is on the other side of this relationship?"

The other side is

```
Role
```

So

```java
inverseJoinColumns = @JoinColumn(name = "role_id")
```

means

```
users_roles

user_id
role_id    ← Other entity (Role)
```

---

## A simple memory trick

Whenever you're inside

```java
class User
```

remember

```
joinColumn
     ↓
Current entity

inverseJoinColumn
     ↓
Other entity
```

---

## Let's reverse the mapping

Suppose instead your mapping was inside `Role`.

```java
public class Role {

    @ManyToMany
    @JoinTable(
        name = "users_roles",
        joinColumns = @JoinColumn(name="role_id"),
        inverseJoinColumns=@JoinColumn(name="user_id")
    )
    private Set<User> users;

}
```

Notice what changed.

Now the current entity is **Role**.

Therefore

```
joinColumn
↓

role_id
```

and

```
inverseJoinColumn
↓

user_id
```

Exactly the opposite.

That's why the word **inverse** is used.

---

# Visual explanation

## Mapping inside User

```
              User Entity
                   |
                   |
         joinColumn=user_id
```

```
                |
                ▼
          users_roles
 --------------------
 | user_id | role_id |
 --------------------
                ▲
                |
inverseJoinColumn=role_id
                |
                |
          Role Entity
```

---

### Mapping inside Role

```
          Role Entity
                |
                |
   joinColumn=role_id
                |
                ▼
          users_roles
 --------------------
 | user_id | role_id |
 --------------------
                ▲
                |
inverseJoinColumn=user_id
                |
                |
          User Entity
```

---

## Why "inverse"?

Think of it like two friends.

```
User  -------------------  Role
       Current             Inverse
```

If you're standing on the **User** side,

```
joinColumn
↓

User
```

```
inverseJoinColumn
↓

Role
```

If you move to the **Role** side,

they swap.

---

## Your code in plain English

When Hibernate reads

```java
@JoinTable(
    name = "users_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
```

it interprets it as:

> "Create a table named `users_roles`. The `user_id` column refers to the current entity (`User`), and the `role_id` column refers to the related entity (`Role`)."

---

### A small note on `referencedColumnName`

Earlier you had:

```java
@JoinColumn(
    name = "user_id",
    referencedColumnName = "user_id"
)
```

Since `user_id` is the primary key of `User`, JPA already assumes that's the referenced column.

So these two are equivalent:

```java
@JoinColumn(name = "user_id")
```

and

```java
@JoinColumn(
    name = "user_id",
    referencedColumnName = "user_id"
)
```

The shorter version is more common unless you specifically need to reference a different unique column.

---

## A trick that helped many developers (including me)

Whenever you forget which is which, ask yourself one question:

> **"Which class am I currently writing?"**

If you're inside `User.java`:

- `joinColumns` → **User's foreign key** (`user_id`)
- `inverseJoinColumns` → **Role's foreign key** (`role_id`)

If you're inside `Role.java`, the answer flips.

That single question is usually enough to determine which column belongs in `joinColumns` and which belongs in `inverseJoinColumns`.

Can you create a QnA using questions I have asked till now for this Spring security and answers you've provided?

Absolutely! I think this is a great idea. You've asked several fundamental questions that every Spring Security developer encounters. I've organized them into a **Q&A notebook** that you can keep for revision.

---

# Spring Security Learning Notes (Q&A)

## Q1. Why was I getting `403 Forbidden` even after configuring a dummy user in `application.properties`?

### Answer

You had configured:

```properties
spring.security.user.name=rahul
spring.security.user.password=password
```

and

```java
http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
```

However, Spring Security didn't know **which authentication mechanism** to use.

Since you were authenticating from Postman using **Basic Authentication**, you needed to enable it explicitly:

```java
http.httpBasic(Customizer.withDefaults());
```

Without this line, Spring Security couldn't process the `Authorization: Basic ...` header.

---

## Q2. Why did I need to disable CSRF?

### Answer

CSRF (Cross-Site Request Forgery) protects applications that use **session-based authentication**.

Example:

```
Browser
   ↓
Login
```

```
   ↓
Session Cookie (JSESSIONID)
```

A malicious website can trick the browser into sending requests with the session cookie.

To prevent this, Spring requires a **CSRF Token**.

---

For REST APIs:

```
React/Postman
      ↓
Authorization Header
      ↓
Basic Auth / JWT
```

Each request carries its own credentials.

There is no browser-managed session.

Therefore, CSRF protection is generally unnecessary.

That's why REST APIs commonly use:

```java
http.csrf(AbstractHttpConfigurer::disable);
```

---

## Q3. Why did my in-memory users return `401 Unauthorized`?

### Answer

You had:

```java
User.withUsername("user")
    .password("p@ssword")
```

Spring Security expects passwords to be **encoded**.

Solutions:

### Learning

```java
.password("{noop}p@ssword")
```

### Production

```java
.password(passwordEncoder.encode("p@ssword"))
```

using

```java
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

---

## Q4. Why do we need a `PasswordEncoder`?

### Answer

Passwords should never be stored in plain text.

Instead of storing:

```
p@ssword
```

Spring stores:

```
$2a$10$MbjOeJmAfQ...
```

This is a BCrypt hash.

During login:

```
Entered Password
        ↓
PasswordEncoder
        ↓
Hash
```

```
        ↓
Compare with Stored Hash
```

Spring never decrypts the password.

It uses

```java
passwordEncoder.matches(rawPassword, encodedPassword)
```

to verify it.

---

## Q5. Authentication vs Authorization

### Authentication

Answers:

> Who are you?

Example:

```
Username: admin
Password: p@ssword
```

---

### Authorization

Answers:

> What are you allowed to do?

Example:

```
ROLE_USER

ROLE_ADMIN
```

Authorization decides which APIs a user may access.

---

## Q6. How can only ADMIN users perform POST operations?

### Answer

Use request matchers.

Example:

```java
.authorizeHttpRequests(auth -> auth

    .requestMatchers(HttpMethod.GET, "/api/employees/**")
    .hasAnyRole("USER","ADMIN")

    .requestMatchers(HttpMethod.POST,"/api/employees")
    .hasRole("ADMIN")

    .requestMatchers(HttpMethod.PUT,"/api/employees/**")
    .hasRole("ADMIN")

    .requestMatchers(HttpMethod.DELETE,"/api/employees/**")
    .hasRole("ADMIN")

    .anyRequest().authenticated()
)
```

Result:

| User | GET | POST | PUT | DELETE |
|---|---|---|---|---|
| USER | ✅ | ❌ | ❌ | ❌ |
| ADMIN | ✅ | ✅ | ✅ | ✅ |

---

## Q7. Difference between `401` and `403`

### 401 Unauthorized

Means:

> Authentication failed.

Examples:

- Wrong password
- Wrong username
- Missing Authorization header

---

### 403 Forbidden

Means:

> Authentication succeeded, but access is denied.

Example:

```
Logged in as USER

↓

Trying ADMIN API

↓

403 Forbidden
```

---

## Q8. Why did Hibernate throw

```
To-one mapping was mapped with targetEntity Role,
but attribute is declared as Set
```

### Answer

The field was:

```java
private Set<Role> roles;
```

but the annotation was a **to-one** relationship (`@OneToOne` or `@ManyToOne`).

Collections must use:

```java
@ManyToMany
```

---

## Q9. Correct Many-to-Many mapping

```java
```

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "users_roles",
    joinColumns = @JoinColumn(name="user_id"),
    inverseJoinColumns = @JoinColumn(name="role_id")
)
private Set<Role> roles = new HashSet<>();
```

---

## Q10. Why should I avoid `CascadeType.ALL`?

### Answer

Roles are **shared**.

Example:

```
Rahul
    ↓
ROLE_USER

Amit
    ↓
ROLE_USER
```

If

```java
CascadeType.ALL
```

includes `REMOVE`, deleting Rahul may also try to delete `ROLE_USER`, which is still needed by Amit.

Recommended:

```java
@ManyToMany(fetch = FetchType.EAGER)
```

or

```java
cascade = {
    CascadeType.PERSIST,
```

```
    CascadeType.MERGE
```

---

## Q11. If I don't use cascade, how will the Role table be populated?

### Answer

Roles are usually **master data**.

Common approaches:

### Option 1

Insert manually

```sql
INSERT INTO roles(name)
VALUES ('ROLE_USER');
```

---

### Option 2 (Recommended)

Seed roles during application startup using `CommandLineRunner`.

---

### Option 3

Use `data.sql`

---

### Option 4

Create Role CRUD APIs (only if roles are managed dynamically).

---

## Q12. Do I need a `RoleRepository`?

### Answer

Yes.

Even if you don't expose Role APIs.

Typical repository:

```java
public interface RoleRepository
        extends JpaRepository<Role, Long>{

    Optional<Role> findByName(String name);

}
```

During user registration:

```java
Role role =
    roleRepository.findByName("ROLE_USER")
                  .orElseThrow();

user.getRoles().add(role);

userRepository.save(user);
```

---

## Q13. Difference between `joinColumns` and `inverseJoinColumns`

Given

```java
@JoinTable(
    name="users_roles",
    joinColumns=@JoinColumn(name="user_id"),
    inverseJoinColumns=@JoinColumn(name="role_id")
)
```

Think from the perspective of the current entity.

If you're inside `User.java`:

```
Current Entity

↓

User

↓
```

```
joinColumns
```

The other entity is

```
Role

↓

inverseJoinColumns
```

Database:

```
users
-------
user_id

roles
-------
role_id

users_roles
-----------------
user_id
role_id
```

- `joinColumns` → Foreign key of the current entity (`User`)
- `inverseJoinColumns` → Foreign key of the related entity (`Role`)

---

## Q14. Should the `Role` entity have a reference to `User`?

### Answer

Not necessarily.

For Spring Security, a **unidirectional** relationship is usually sufficient:

```
User
  |
  ▼
Role
```

You only need the reverse mapping if your application needs to retrieve all users assigned to a specific role.

---

### Q15. Why is `FetchType.EAGER` commonly used for roles?

#### Answer

During authentication, Spring Security needs the user's roles immediately to determine what the user is allowed to access.

With `FetchType.EAGER`, when a user is loaded, the associated roles are fetched automatically.

This makes converting roles into `GrantedAuthority` objects straightforward during authentication.

---

## Suggested Learning Roadmap

You've completed:

- ✅ Basic Authentication
- ✅ Security Filter Chain
- ✅ CSRF
- ✅ Password Encoding
- ✅ In-Memory Users
- ✅ Authentication vs Authorization
- ✅ Role-Based Authorization
- ✅ User–Role Many-to-Many Mapping
- ✅ `joinColumns` vs `inverseJoinColumns`
- ✅ Understanding `CascadeType`
- ✅ Planning Role Seeding

Next topics I'd recommend:

1. Custom `UserDetails`
2. Custom `UserDetailsService`
3. Loading users and roles from the database
4. Converting roles to `GrantedAuthority`
5. Login flow using the database
6. JWT authentication
7. Refresh Tokens
8. Method-level security (`@PreAuthorize`)
9. Exception handling (`AuthenticationEntryPoint`, `AccessDeniedHandler`)

---

I think this has become a solid set of notes already. As you continue learning Spring Security (database authentication, JWT, OAuth2, etc.), we can keep expanding this into a complete interview and revision handbook.